Team : NEVER PAUSE UNIT

NPU Challenge

NPU : Never Pause Unit 연구 진행 발표

박준범 윤수민 최민기 최윤성 | 2026.07.14

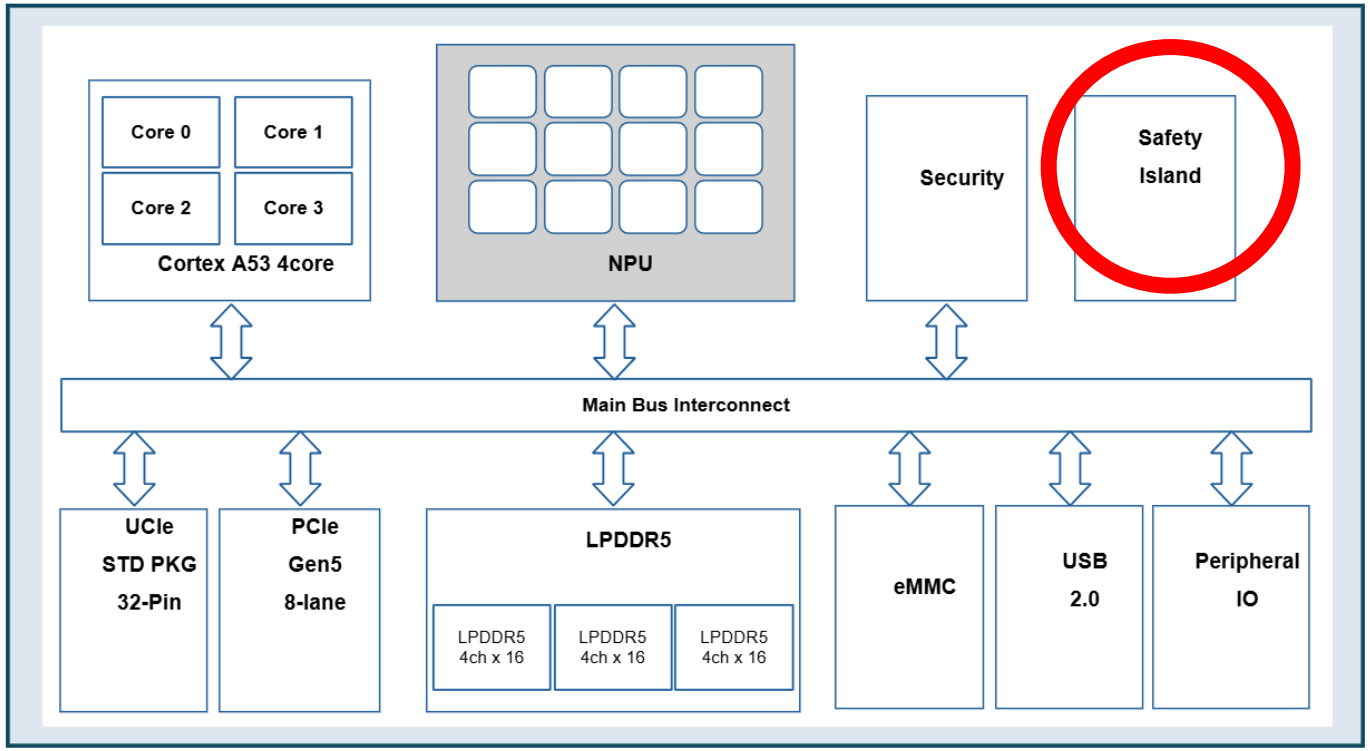
Overview

- 주제 구체화
- Week 4 진행 사항 및 Week 5 계획

Introduction

- NPU 코어로 텐스토렌트의 tensix 코어를 활용 하였다.
- 데이터센터 아키텍처와 차량용 ai 반도체 사이에는 안전성에 대한 Gap 존재한다.
- Saftety Island 존재하지만 SDC (Silent Data Corruption) 찾아내기엔 한계가 존재한다.

▪ SoC Block Diagram



https://github.com/tenstorrent/tt_metal/tt-llk/docs/llk/l2/images/tensix_engine.png

SDC (Silent Data Corruption)

1. Fault 분류 (결함 표출)

- Masked
- Signaled/Detected Error
- Hang/Crash
- SDC

SDC는 결함을 알 수 없어 잘못된 결과를 내뱉어도 확인이 불가능하다

(차량용 반도체에게 있어 큰 문제)

2. SDC 분류 (결함 지속성)

- Transient Fault
- Permanent Fault

1. 특성별 다른 검증 방식
2. Transient Fault가 반복되면 해당 연산 유휴 코어로 매핑 (가상 코어 활용)

Transient Fault 특성별 검증 방식

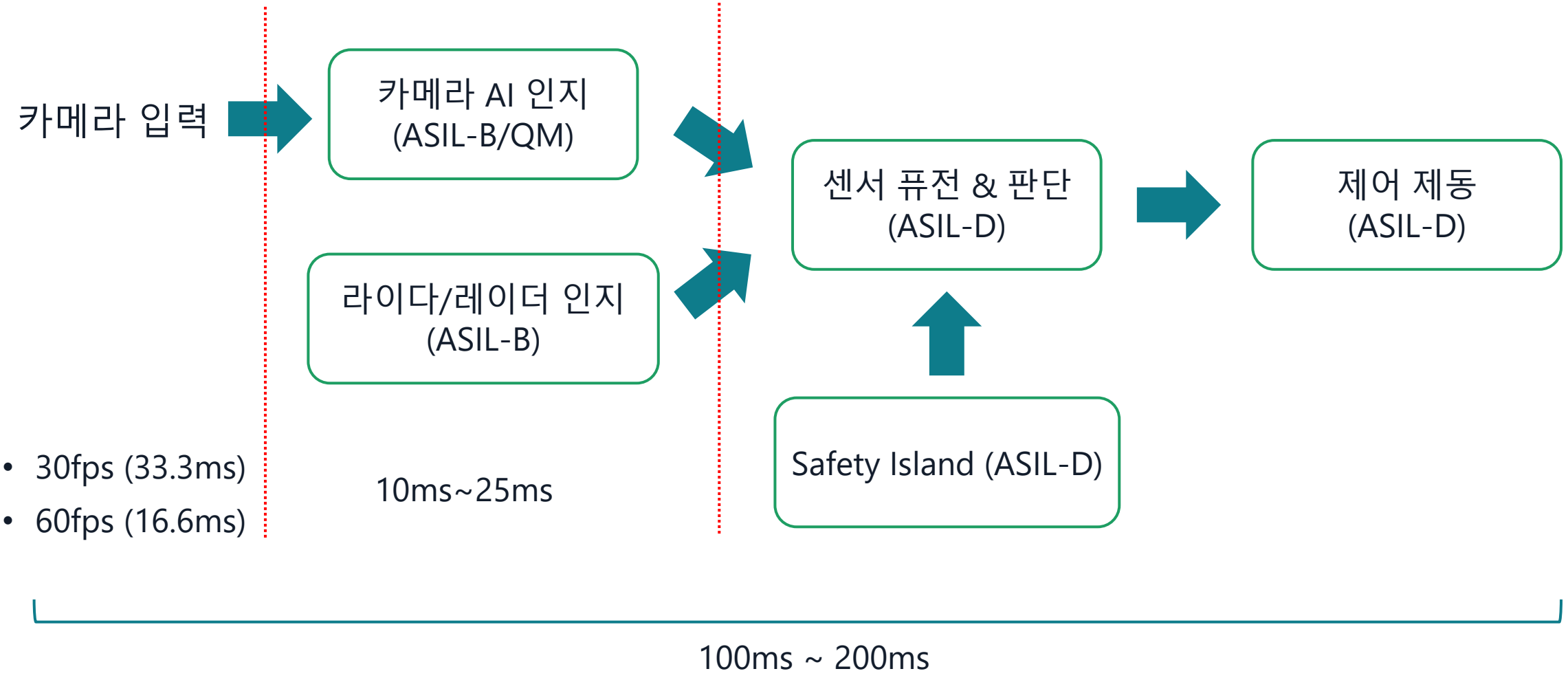
Compute-time (FPU/SFPU 연산 오류)	**Storage-time** (L1 데이터 저장오류)	**Transport-time** (NoC payload 전송 오류)
재연산 후 비교	Checksum 유지 및 주기적인 검사	Payload에 함께 보낸 checksum 검사

모든 연산을 재연산하거나 모든 비트에 대해 checksum 검사를 진행하는 것은 성능 손실이 과대할 것이다.

Compute-time → 중요 레이어에 대해 재연산 후 비교

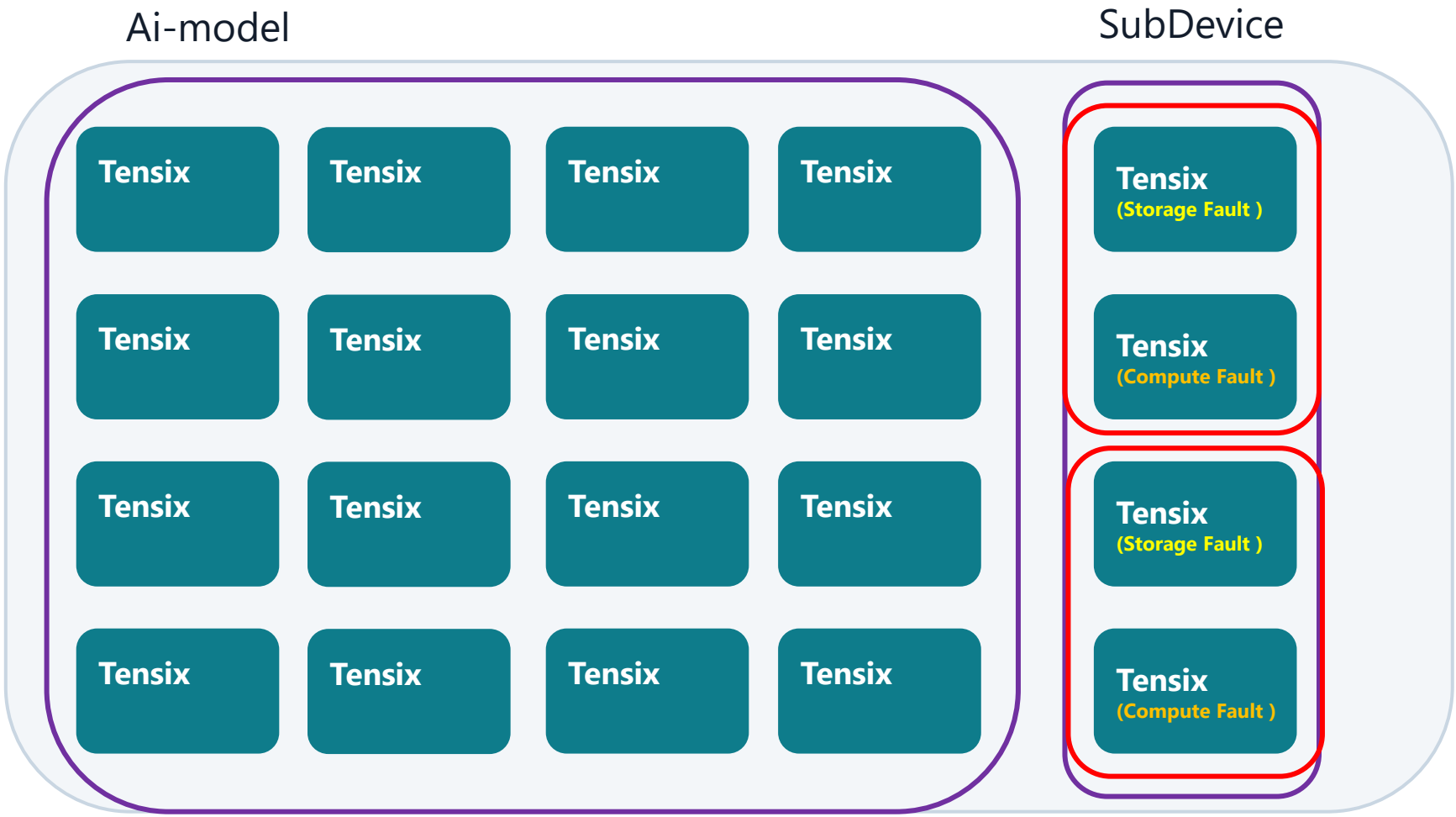
Storage-time → 지수 비트에 대해서만 주기적인 검사

BOS 반도체의 방식



https://github.com/tenstorrent/tt-metal/tree/main/docs/source/common/images

Idea



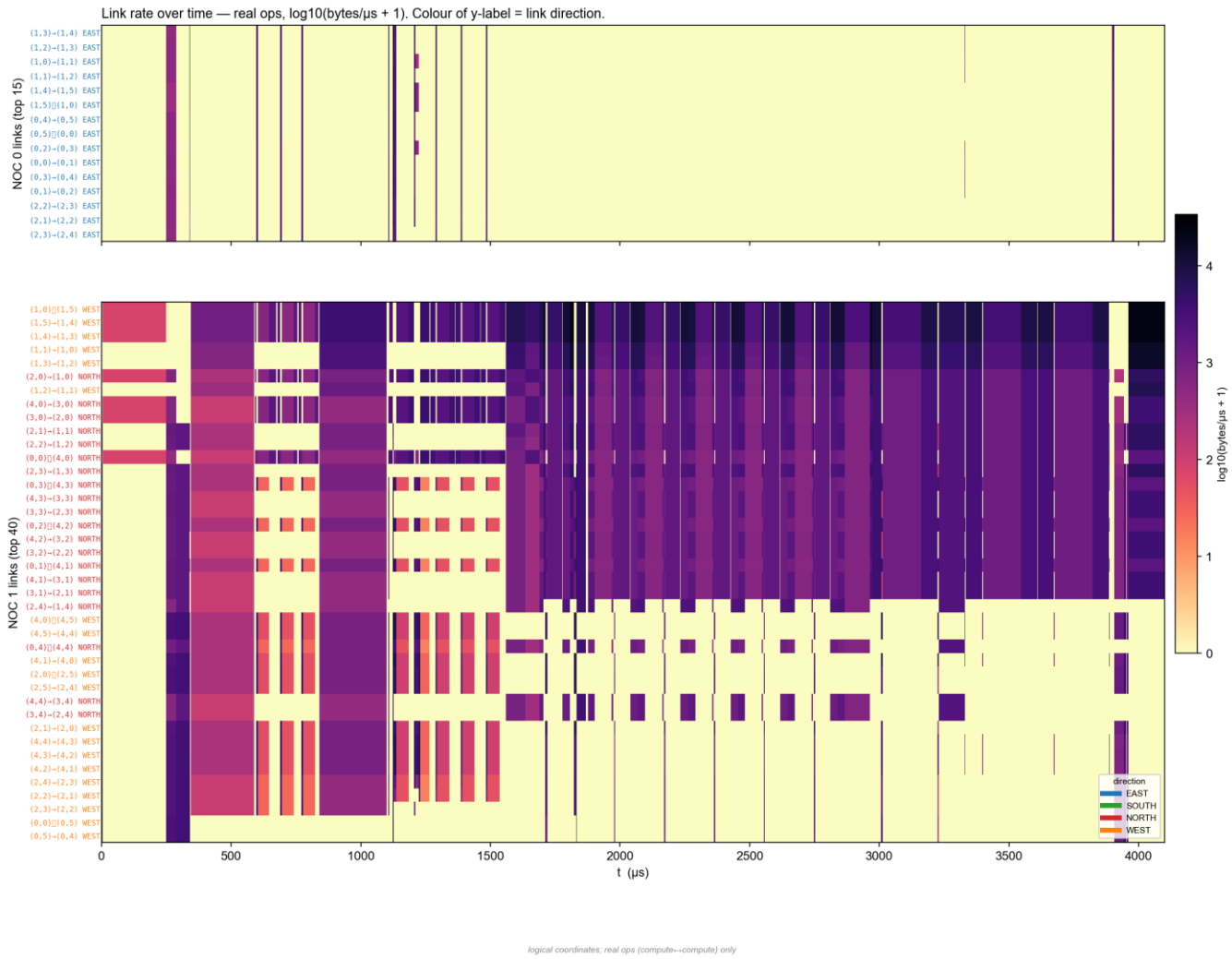
하드웨어 추가 없이 유휴코어만을 활용하여 NPU 칩 자체에서 fault tolerant하게 구현 해보겠다.

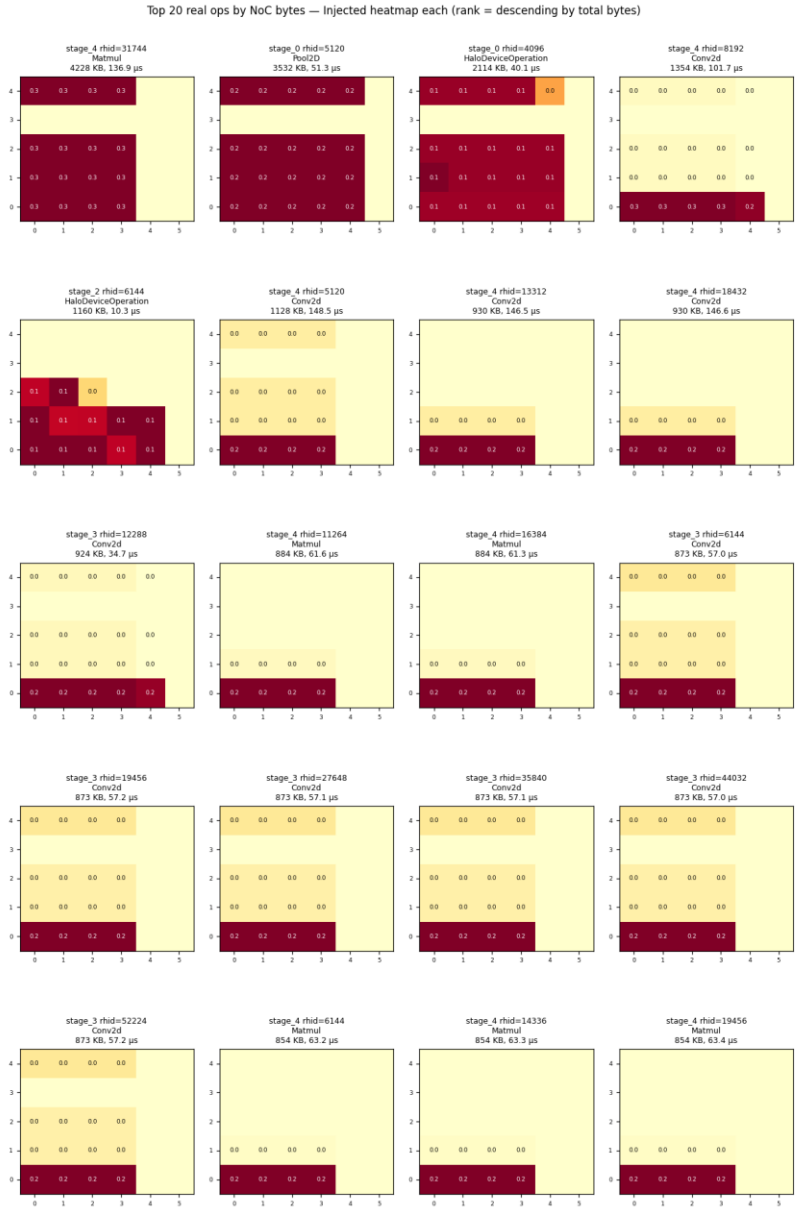
https://github.com/tenstorrent/tt-metal/tree/main/docs/source/common/images

NoC traffic analysis (NPE)

NoC0 (Read)

NoC1 (Write)





Week 4
Subdevice 구동을 위한 코어 제한

1. 16 core 사용으로 제한
2. 코어 제한 결과 성능 비교

코어 제한 (conv2d, 14 core)

- 121. ttnn.conv2d

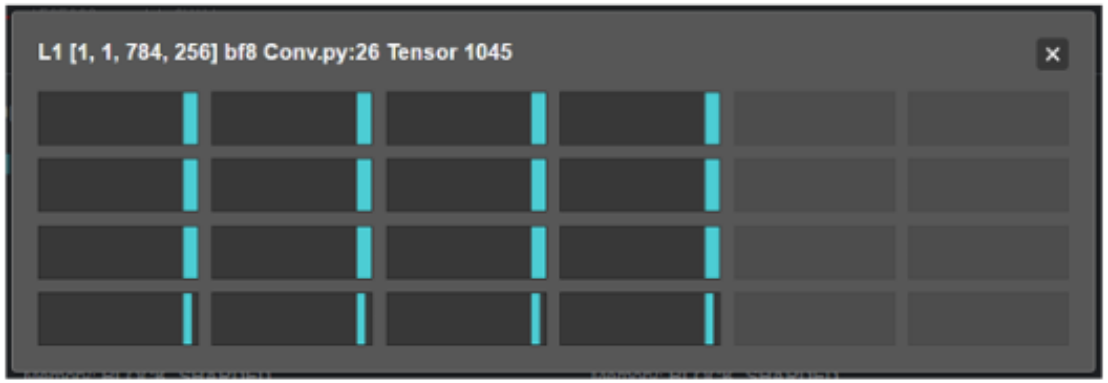


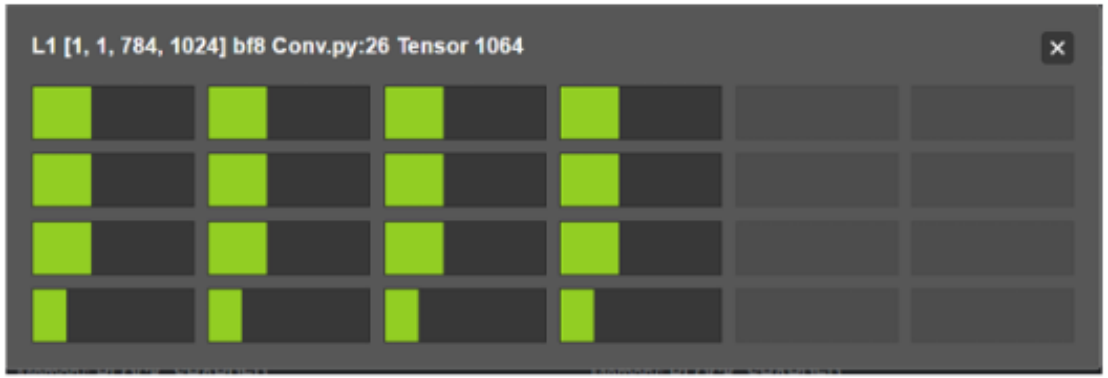
- 121. ttnn.conv2d



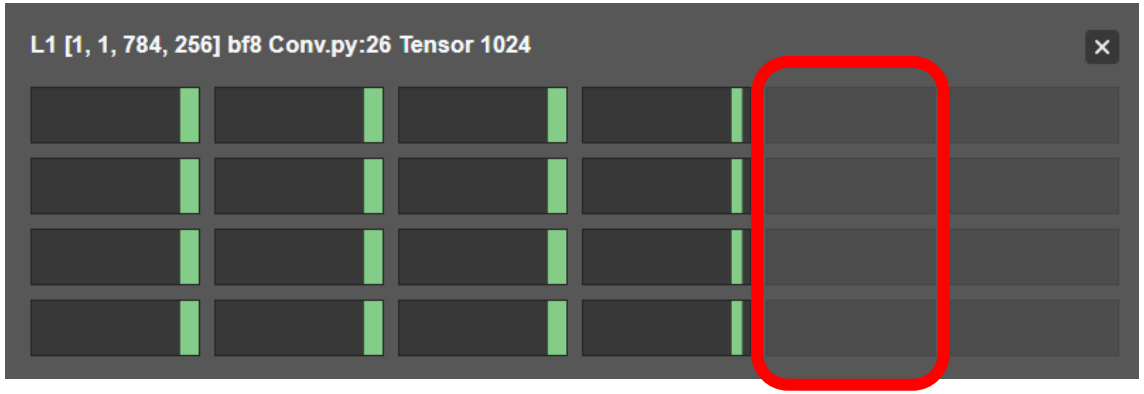
코어 제한 (conv2d, 16 core)

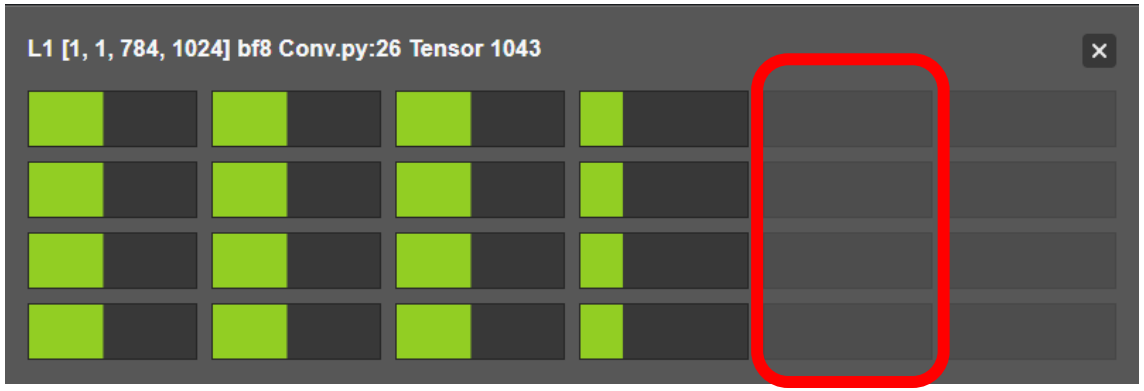
- 197. ttnn.conv2d





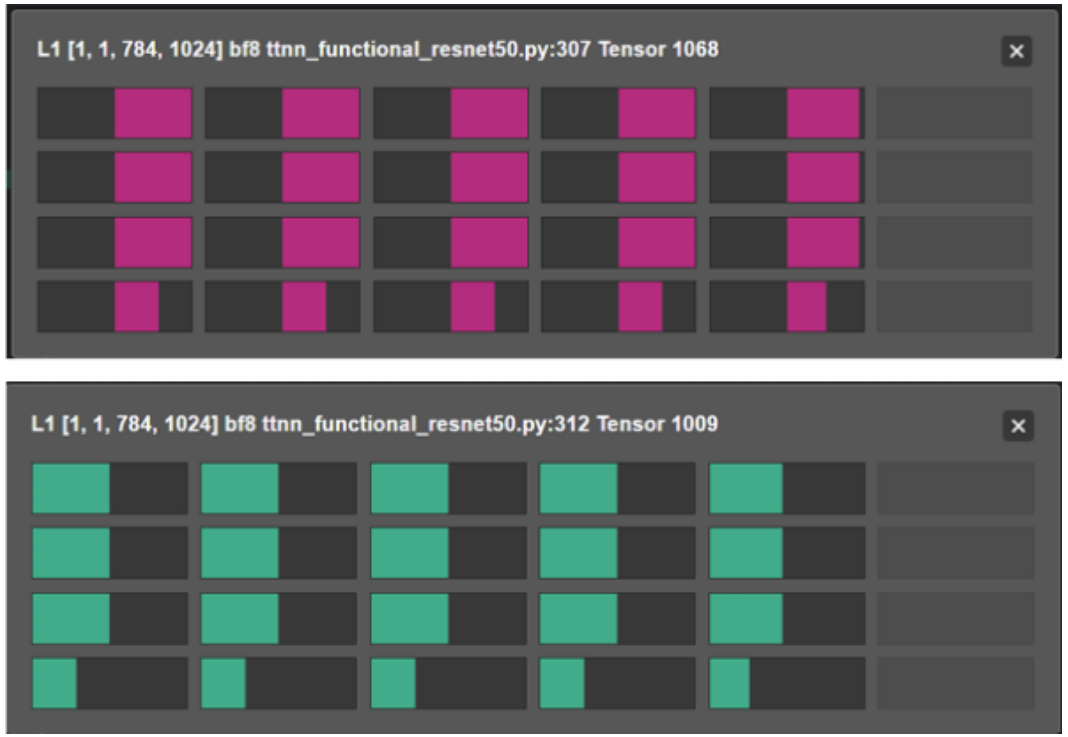
- 191. ttnn.conv2d





코어 제한 (add, 20 core)

- 199. ttnn.add

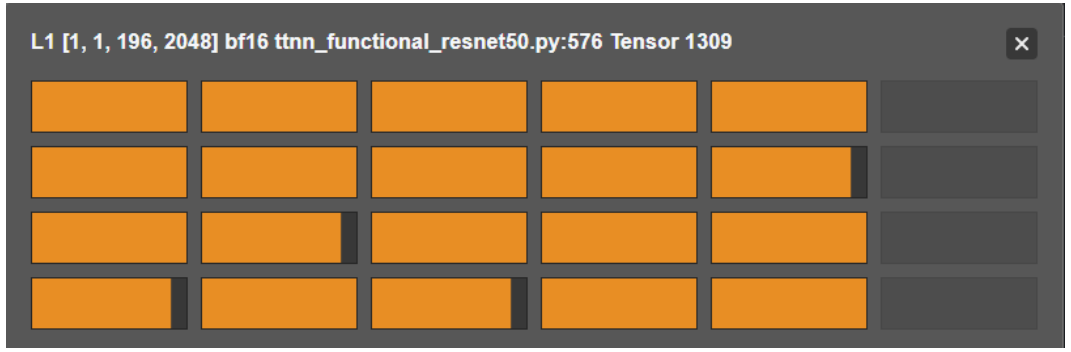


- 192. ttnn.add

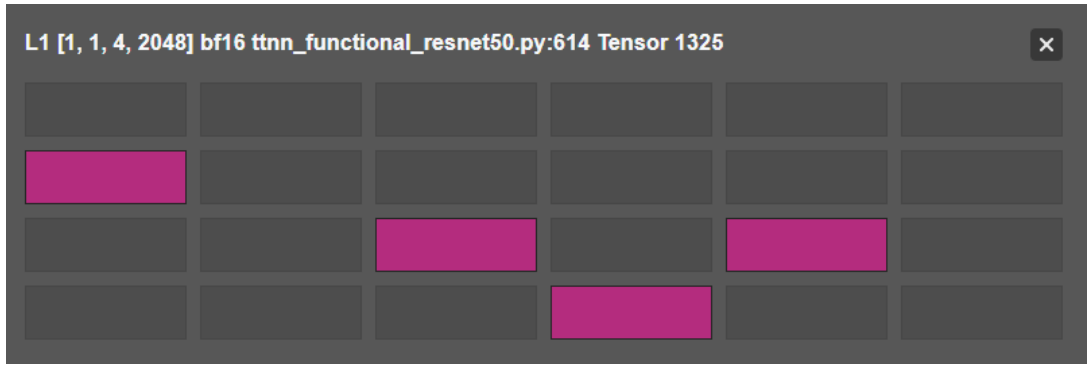


코어 제한 (reshape, 4 core)

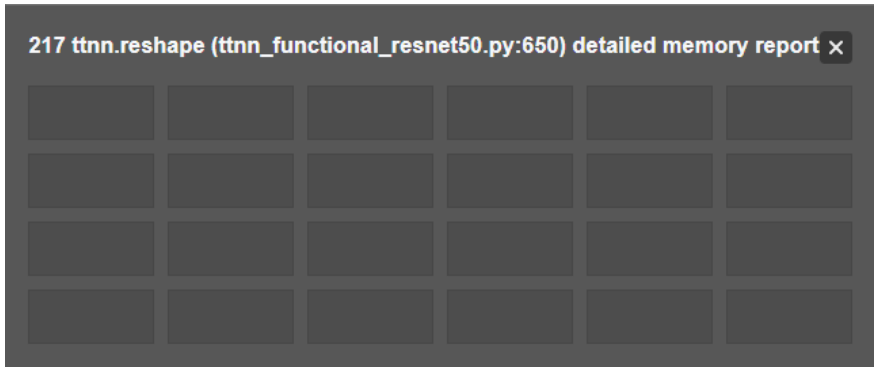
- 226. ttnn.reshape



- 232. ttnn.reshape



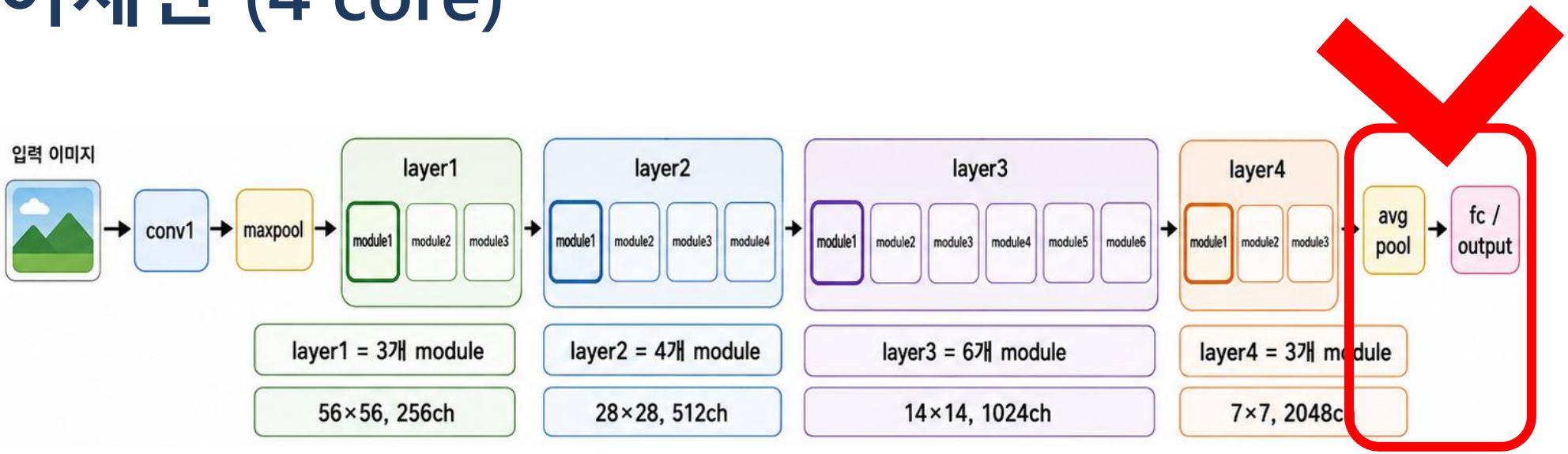
- 217. ttnn.reshape



- 228. ttnn.reshape



코어제한 (4 core)

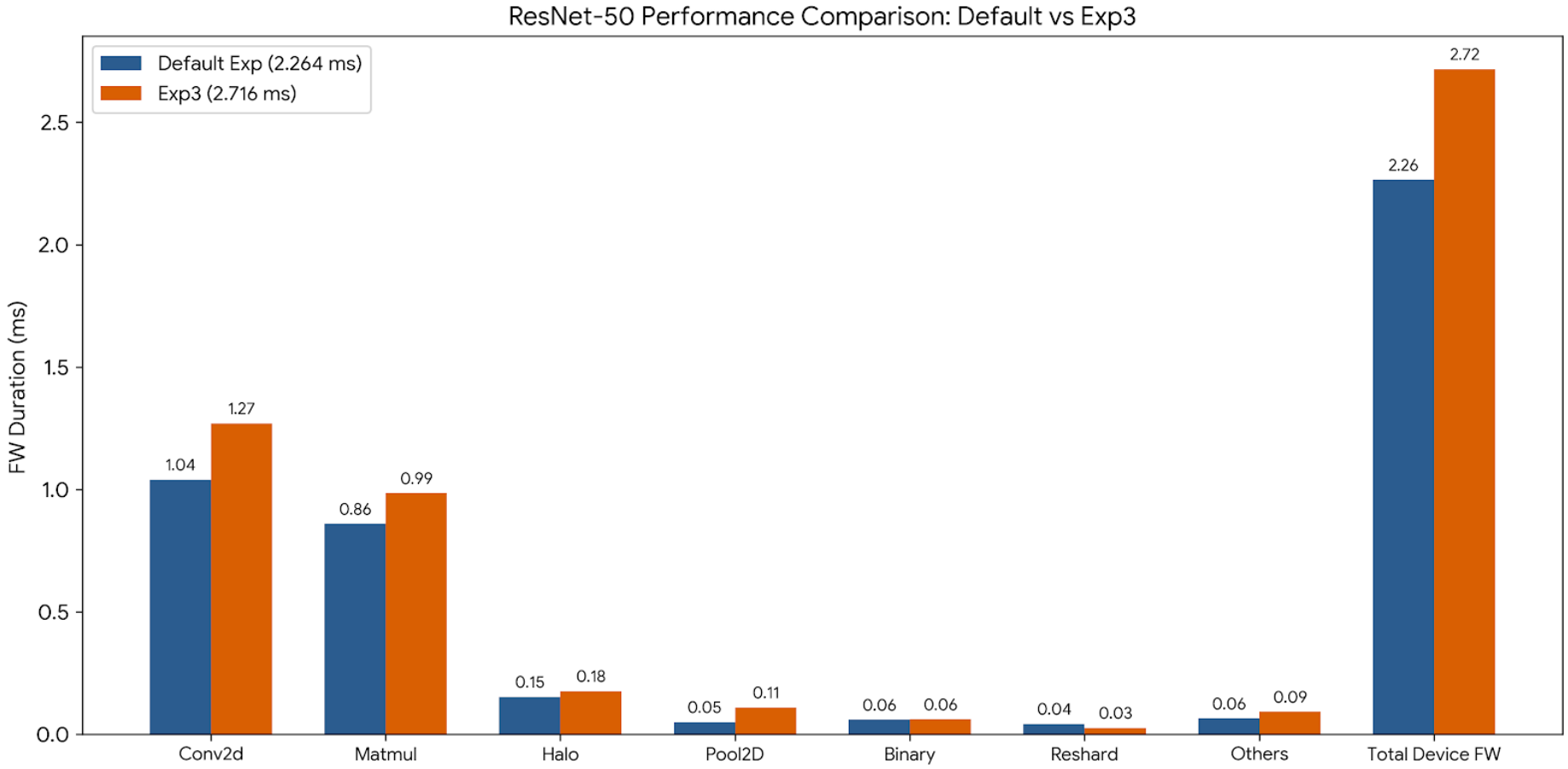


228 ttnn.tilize_with_val_padding (ttnn_functional_resnet50.py:598) Memory details Source
229 ttnn.global_avg_pool2d (ttnn_functional_resnet50.py:607) Memory details Source
230 ttnn.untilize_with_unpadding (ttnn_functional_resnet50.py:610) Memory details Source
231 ttnn.to_memory_config (ttnn_functional_resnet50.py:613) Memory details Source
232 ttnn.reshape (ttnn_functional_resnet50.py:614) Memory details Source
233 ttnn.to_memory_config (ttnn_functional_resnet50.py:615) Memory details Source
234 ttnn.tilize_with_val_padding (ttnn_functional_resnet50.py:635) Memory details Source
235 ttnn.linear (Linear.py:22) Memory details Source
236 ttnn.untilize_with_unpadding (ttnn_functional_resnet50.py:654) Memory details Source
237 ttnn.reshape (ttnn_functional_resnet50.py:657) Memory details Source

Reshape 전 Sharding에서 **Interleaving** 방식으로 전환할 때 **코어 제한이 풀리는** 오류 발생

→ **Interleaving으로 바꾸는 대신 DRAM으로 보냈다 다시 가져옴**

코어제한 결과 비교



코어 제한시 약 20% 성능 하락 (순수 디바이스 수행시간 2.264 ms → 2.716 ms)

코어제한 결과 비교

OP Code	호출 횟수 (Base → 제한)	Default FW (ms)	Exp3 FW (ms)	증감 (ms)	변동률 (%)
Conv2d	20 → 20	1.038 ms	**1.269 ms**	**+0.231 ms**	**+22.2%**
Matmul	34 → 34	0.860 ms	**0.985 ms**	**+0.125 ms**	**+14.5%**
HaloDeviceOperation	21 → 21	0.153 ms	**0.176 ms**	**+0.023 ms**	+15.3%
Pool2D	1 → 1	0.049 ms	**0.108 ms**	**+0.058 ms**	**+117.6%**
BinaryDeviceOperation	16 → 16	0.059 ms	**0.061 ms**	+0.001 ms	+2.1%
ReshardDeviceOperation	18 → **9**	0.041 ms	**0.025 ms**	**-0.015 ms**	**-38.0%**
기타 연산	13 → 14	0.064 ms	**0.092 ms**	+0.028 ms	-
전체 합계	**123 → 115**	**2.264 ms**	**2.716 ms**	**+0.452 ms**	**+20.0%**

Conv2d, matmul 연산이 지연시간 상승의 약 79% 차지 한다.

코어제한 결과 비교

OP Index	연산 위치	Input Feature Map Shape	Core Count (Base ➔ 제한)	FW Duration (Default)	FW Duration (Exp3)	지연 시간 (Diff)	비중 (%)
OP 0	**바틀넥 진입 전 (Conv1)**	**115 × 115 × 16**	**20개 ➔ 9개**	**99.9 µs**	**212.6 µs**	**+112.7 µs**	**48.8%**
OP 10	**바틀넥 Stage 2 Projection**	28 × 28 × 512	20개 ➔ 16개	31.1 µs	77.2 µs	**+46.1 µs**	**20.0%**
OP 17	**바틀넥 Stage 3 Projection**	14 × 14 × 1024	20개 ➔ 16개	90.6 µs	133.0 µs	**+42.4 µs**	**18.4%**
기타	나머지 17개 Conv2d 연산	다양한 크기	-	785.8 µs	815.6 µs	+29.8 µs	12.9%
합계	**Conv2d 전체 (20개)**	-	-	**1,038.4 µs**	**1,269.4 µs**	**+231.0 µs**	**100.0%**

OP 0가 Conv2d 연산 지연시간 증가분의 48.8%를 차지한다.

향후 계획

~~1주차 · HW 아키텍쳐 학습~~

~~2주차 · SW 스택 이해~~

~~3주차 · AI model 포로파일링 및 연구주제 탐구~~

~~4주차 · 여유 코어 확보 및 SubDevice 탐구~~

5주차 ·
여유 코어에 서브디바이스 적용(Fault detect logic)
- 연산 결과에 치명적인 요소들 탐색

코어 제한 모델 최적화
- 베이스라인 모델 수준으로 최적화